

Processor Selection for Embedded Systems

Most embedded systems need some type of processor, but which one would you select? Would you go for the one that is familiar because it is what 'we have always used,' or would you benchmark them with your application code first? Well, it could be a mix of both and then some others, which this article will help you determine

PANKAJ V. AND DILIN ANAND

When we began working on this article, the first question in our mind was about what made it worthwhile to spend time on this. A few questions asked to some of my knowledgeable friends in system design gave us our first answer for the story.

For starters, it is the best way to reduce power consumption. Simply moving a generation ahead on a manufacturing process node gives you vastly reduced power consumption figures, or more power for the same performance—if that is what you want. This, in turn, reduces the heat generated by the processor, which means smaller heat sinks that allow you to get into an even smaller casing, or even go fanless.

On the other hand, you could select the processor such that it can handle all your number crunching and polygon-rendering madness. This lets you ensure a smooth UI, get new functionality in or go for cost-effectiveness.

Overall, the right selection helps you enhance the user experience by reducing the chance of system overheating like what happened in some of the recent mobile devices, or any other untoward event that might make your customer curse the designer.

Where do we begin

Apart from which central processing unit (CPU) to select, there is also the problem of whether you should have gone for a graphics processing unit (GPU) rather than a CPU. So let us find out which one works best for you.

A CPU is like an executive that keeps on switching between tasks, directing IOs and peripherals to perform their operations and handling virtual memory functions. The GPU, on the other hand, is like a factory employee that can handle repetitive tasks with flair. It cannot switch tasks as fast as the CPU, but it has a lot more arithmetic logic units (ALUs) that make it better at performing mathematical tasks.

"Most CPUs have an execution unit with branch or loops capability, which is used for controlling other logic blocks. GPUs are meant for highly parallel calculation, where data streams are operated upon, not for control application like a normal CPU is," explains Satish Bagalkotkar, president and CEO, Synapse Design Automation.

The spokesman goes on to explain that GPUs are optimal for tight code with high parallelisation since they have hundreds of simple execution units with SIMD (single instruction, multiple data) capabilities. Algorithms that can be parallelised easily are best implemented on GPUs. A GPU normally has smaller memory availability in normal implementation than a CPU, so a GPU is probably not well suited for a data-intensive application.

Typically, a GPU assists video rendering for a CPU, but they also have some additional applications. Earlier servers did not need GPUs as they were managed over a text-based interface. However, firms have already announced GPU-based servers. Some of the key markets where CPU and GPU coexist are mobile devices such as smartphones and tablets, financial trading, video processing, medical imaging, etc. Some

of the high-end GPUs with a control CPU are extensively used in industrial simulations, games, entertainment and video processing. There is also an increased trend of high-performance GPU adoption in the finance area.

How does a GPU get more work done, whereas CPUs run at a higher frequency? We guess an example could be a case of Bitcoin mining. The four cores of a CPU like the Intel i7-4770k, albeit running at a higher clock of 3.5 GHz, will be able to execute far less instructions per clock than a GPU like AMD Radeon R9 290X, which has 2816 stream processors at a comparatively lower clock of 1000 MHz.

How to select a GPU

Within GPUs, you can choose between solutions that have more complex shaders and those with simple shaders—the difference being that the GPUs with simple shaders will pack a lot more into the same die space. But which one do you need?

"It is really application dependent. The real differentiator is whether the shader implements the functions and calculations needed by the application being discussed. If the shader does not implement the specific instructions needed, the complexity is irrelevant. From the design point of view, a simple shader is easy to implement and replicate to meet the performance," says Satish.

How to select a CPU

The most important consideration for selecting a CPU is the application itself. Each embedded application comes with a specific set of requirements.